

Lab 4 — Batch B

Java · 15:30–16:55

Language: Java

(Logistics: compile with `javac *.java` and run with `java <ClassWithMain>`. Ask a TA early if you're stuck on the mechanics of this rather than losing time to it.)

Q1 (3 marks) — Design: a `DataConsumer` hierarchy

Your hostel WiFi has a monthly data cap, and you want to estimate how much of it different activities on your devices will use up. Design a small class hierarchy for this.

(a) [0.5] Define an interface (or abstract class, your choice) `DataConsumer` with a method `double dataUsageGB()`.

(b) [1] Implement two concrete classes, each with **properly encapsulated fields** (not `public`) and a constructor:

- `VideoStreamer`, with `hoursWatched` and `gbPerHour` fields — `dataUsageGB()` returns `hoursWatched * gbPerHour`
- `FileDownloader`, with `numFiles` and `avgFileSizeGB` fields — `dataUsageGB()` returns `numFiles * avgFileSizeGB`

(c) [0.5] Implement `HDVideoStreamer`, which **extends** `VideoStreamer`. Its constructor should take `hoursWatched`, `gbPerHour`, and `extraGB` (the extra data an HD stream burns beyond the standard rate), calling `super` appropriately, and its `dataUsageGB()` should return the inherited usage *plus* `extraGB` (via `super.dataUsageGB()`, not by duplicating the base calculation).

(d) [0.5] Write a method `totalDataUsage(DataConsumer[] devices)` that returns the sum of usage across an array of devices. It must work correctly regardless of which concrete device types are in the array — **do not** use `instanceof` or check for specific class types anywhere in this method; it should work purely by calling `device.dataUsageGB()` and trusting each device to know its own usage.

(e) [0.5] Write a short `main` method that creates at least one `VideoStreamer`, one `FileDownloader`, and one `HDVideoStreamer`, puts them all in a single `DataConsumer[]`, and prints the result of `totalDataUsage` on that array.

(You may recognize this as basically the same idea as the tagged `real-part` /generic-dispatch story from a couple of lectures ago — the difference is that here, Java's own method dispatch does the "which implementation do I call" work for you, instead of you writing a `cond` /tag-checking dispatch procedure by hand. Part (d)'s ban on `instanceof` is there for exactly this reason — falling back to type-checks defeats the entire point of using an interface.)

Take-home: some things worth wondering about (ungraded, no code needed)

You don't need to write anything for these — just think about them, and look things up if you're curious. A sentence or two of your own conclusion is worth more than a long researched answer you don't actually understand.

1. How does Java actually *do this*? In Scheme, we made dispatch happen ourselves — either an explicit `cond` checking a tag, or a closure that responds to messages. In today's exercise, you never wrote anything that says "if this is a `VideoStreamer`, do X; if it's a `FileDownloader`, do Y" — and yet `device.dataUsageGB()` still calls the right one every time. Something has to be making that decision at runtime. Look up **virtual method tables (vtables)** and see if you can explain, in your own words, roughly what's happening the moment `device.dataUsageGB()` executes.

2. Why can a class *implement* many interfaces, but *extend* only one? `HDVideoStreamer` extends `VideoStreamer` — Java would refuse to let it also extend `FileDownloader`. But it's completely happy for a single class to *implement* five different interfaces at once. Why the difference? Look up the **"diamond problem"** in multiple inheritance (a classic issue in languages like C++ that *do* allow a class to extend two classes) and see if you can explain what specifically goes wrong, and why interfaces mostly sidestep it.

Once you're done, feel free to also try Batch A's Q1 for extra practice — same topics, different problem.